

ANKARA UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING



COM 4532
BLOCKCHAIN TECHNOLOGY AND PUBLIC LEDGERS
PROJECT PROPOSAL REPORT

Multi-Signature Wallet Implementation

Merve ÇAKIR-21290542

Emine BİNAY-22290370

İrem KESKİN-21290538

Dr. Murat OSMANOĞLU

04.05.2026

ABSTRACT

This project presents the design, implementation, and evaluation of a smart contract-based multi-signature wallet system deployed on the Ethereum blockchain. Single-signature wallets, which rely on a single private key for transaction authorization, constitute a critical single point of failure that becomes increasingly problematic as the value and complexity of digital asset holdings grow. The proposed system addresses this vulnerability by distributing transaction authorization among multiple parties through a configurable M-of-N threshold signature scheme, eliminating the need for any centralized intermediary while preserving the core blockchain properties of trustlessness, transparency, and censorship resistance.

The implementation is structured across two complementary layers. The on-chain layer consists of modular Solidity smart contracts governing wallet ownership, signature threshold enforcement, and the complete transaction lifecycle from proposal to execution. The off-chain layer is developed in Python using Web3.py for blockchain interaction and Streamlit for the user-facing dashboard, enabling wallet owners and co-signers to propose, review, and approve transactions without requiring low-level blockchain expertise. Security measures include nonce-based replay protection compliant with EIP-155, reentrancy mitigation through the Checks-Effects-Interactions pattern and OpenZeppelin security libraries, time-lock mechanisms for high-value and administrative transactions, and hard-coded minimum threshold enforcement against quorum manipulation. Gas cost optimization is pursued through off-chain signature aggregation using EIP-712 typed structured data.

The evaluation strategy covers three dimensions: functional correctness through automated unit and integration tests on a local Hardhat network targeting greater than 90% code coverage, security verification through Slither and Mythril static analysis tools supplemented by a manual audit, and gas efficiency benchmarking against Gnosis Safe as the current industry standard. The ultimate objective is to deliver a working prototype that demonstrates decentralized, transparent, and programmable multi-party asset management remains achievable without sacrificing usability or security.

TABLE OF CONTENTS

ABSTRACT	ii
TABLE OF CONTENTS	iii
1. INTRODUCTION	1
1.1 Real-World Problem Description	1
1.2 Current Solutions and Their Limitations	1
1.2.1 Centralized Custody Solutions	1
1.2.2 Existing Blockchain-Based Multi-Signature Solutions	2
1.2.3 Hardware Wallet Multi-Signature	3
1.2.4 Comparative Analysis	3
1.3 Why Blockchain is Suitable for Multi-Signature Wallets	4
1.3.1 Trustlessness and Elimination of Intermediaries	4
1.3.2 Transparency and Complete Auditability	4
1.3.3 Programmable Security Policies and Governance	5
1.3.4 Censorship Resistance and Permissionless Access	6
1.3.5 Cryptographic Security and Immutability	6
1.3.6 Cost Efficiency	7
1.3.7 Composability with Decentralized Finance (DeFi)	7
1.3.8 Alignment with Blockchain Philosophy	7
1.4 Scope and Objectives	8
2. PROBLEM FORMULATION	10
2.1 Stakeholders and Roles	10
2.2 System Interactions and Workflow	11
2.2.1 Wallet Creation Flow	11
2.2.2 Transaction Flow	12
2.2.3 Administrative Operations	12
2.3 Security Concerns and Attack Vectors	12
2.4 Scalability and Gas Optimization:	14
3. PROPOSED METHOD / APPROACH	16
3.1 Smart Contract Architecture	16
3.2 Platform Components	16
3.3 Tools and Frameworks	17
3.4 Key Features	17
4. EVALUATION PLAN	19
4.1 Functional Correctness Testing	19
4.2 Security Evaluation	19
4.3 Gas Efficiency Analysis	20
5. CONCLUSION	21
REFERENCES	22

1. INTRODUCTION

1.1 Real-World Problem Description

The rapid adoption of blockchain technology and cryptocurrencies has created unprecedented opportunities for decentralized financial systems. However, this growth has also exposed critical vulnerabilities in traditional cryptocurrency wallet management. Single-signature wallets, which rely on a single private key for transaction authorization, represent a fundamental security weakness that becomes increasingly problematic as the value and complexity of digital asset holdings grow [1].

The core problem is straightforward: **a single private key constitutes a single point of failure**. If this key is lost, stolen, or compromised, the consequences are immediate and irreversible [2]. Unlike traditional banking systems where account recovery mechanisms exist, cryptocurrency transactions are final and irreversible by design. This characteristic, while ensuring transaction immutability and censorship resistance, creates severe risks for individual users and organizations managing substantial cryptocurrency holdings.

Consider the real-world implications: organizations such as Decentralized Autonomous Organizations (DAOs) collectively manage billions of dollars in digital assets. Corporate treasuries increasingly hold cryptocurrency as part of their balance sheets, with companies like MicroStrategy holding over 140,000 BTC [3]. Family estates include digital assets requiring secure inheritance mechanisms. In all these scenarios, entrusting complete control to a single individual or private key creates unacceptable risks of loss, theft, internal fraud, and operational disruption.

Furthermore, single-signature wallets provide no accountability or governance mechanisms. When a transaction is executed, there is no record of who authorized it, no approval workflow, and no ability to implement spending limits or role-based access controls [4]. For organizations requiring transparent financial management and regulatory compliance, these limitations are untenable.

The fundamental challenge this project addresses is: How can multiple parties securely co-manage cryptocurrency funds while maintaining the core benefits of blockchain technology—decentralization, transparency, and trustlessness—without introducing centralized intermediaries or single points of failure?

1.2 Current Solutions and Their Limitations

Several approaches currently exist for multi-party cryptocurrency fund management, each with distinct trade-offs between security, usability, and decentralization.

1.2.1 Centralized Custody Solutions

Coinbase Custody and **BitGo** represent the institutional approach to multi-party fund management [5]. These services provide professional-grade security infrastructure, insurance coverage, and regulatory compliance features that appeal to enterprises and institutional investors.

Advantages: User-friendly interfaces similar to traditional banking, insurance protection (Coinbase Custody offers up to \$255 million coverage), and compliance with regulatory requirements including Know Your Customer (KYC) and Anti-Money Laundering (AML) procedures.

Critical Limitations: These solutions fundamentally contradict blockchain's core principle of decentralization [6]. Users must trust the custodian's security practices, operational integrity, and financial solvency. The custodian controls private keys, meaning users do not maintain true ownership of their assets—a principle summarized in the cryptocurrency community as "not your keys, not your coins" [7]. Additionally, custodial services face regulatory risks, including potential government seizure or account freezing. Ongoing custody fees typically range from 0.5% to 1% annually, creating significant costs for long-term holdings.

1.2.2 Existing Blockchain-Based Multi-Signature Solutions

Gnosis Safe has emerged as the dominant smart contract-based multi-signature wallet on Ethereum, managing over \$40 billion in assets across multiple blockchain networks [8]. Gnosis Safe implements M-of-N signature schemes directly in smart contracts, allowing organizations to require multiple approvals for transaction execution.

Advantages: True decentralization with no trusted intermediaries, transparent on-chain transaction history, modular architecture supporting extensions and integrations, and battle-tested security through years of production use and multiple professional audits.

Limitations: Despite significant improvements, Gnosis Safe presents usability challenges for non-technical users. The initial setup requires understanding Ethereum transactions, gas fees, and wallet interactions. On Ethereum mainnet, transaction costs can be substantial—often \$50-200 per transaction during network congestion. While the platform offers customization through modules, implementing specific governance models may require custom smart contract development.

Bitcoin's Native Multi-Signature capability, implemented through P2SH (Pay-to-Script-Hash) addresses, has been part of the Bitcoin protocol since 2012 [9]. This provides protocol-level support for M-of-N signature schemes without requiring additional smart contract layers.

Advantages: Deep integration with Bitcoin's core protocol, extensive battle-testing over a decade, broad wallet compatibility, and no smart contract complexity or vulnerabilities.

Limitations: Bitcoin's limited scripting capabilities prevent implementation of advanced governance features such as time-locks, conditional logic, spending limits, or dynamic threshold adjustments [10]. Users typically interact through command-line tools or specialized wallet software, creating usability barriers. Additionally, multi-signature transactions are larger than standard transactions, resulting in higher transaction fees.

1.2.3 Hardware Wallet Multi-Signature

Ledger and **Trezor** hardware wallets support multi-signature configurations where multiple physical devices must approve transactions. This approach emphasizes security through physical device isolation.

Advantages: Private keys never leave hardware devices, providing strong protection against malware and remote attacks. No reliance on third-party custody services maintains user sovereignty over assets.

Limitations: Setup complexity requires significant technical expertise. Coordinating multiple hardware devices across different physical locations introduces logistical challenges. The approach provides only basic M-of-N signature functionality without advanced governance features like automated workflows or programmable rules. Finally, if multiple devices are lost or damaged, fund recovery may be impossible despite the multi-signature configuration.

1.2.4 Comparative Analysis

The following table summarizes the key trade-offs among existing solutions:

Table 1: Comparison of Existing Multi-Signature Wallet Solutions

Solution Type	Decentralization	Security	Usability	Programmability	Cost
Coinbase Custody	Low	High	High	Low	High (0.5-1% annual)
BitGo	Medium	High	Medium	Medium	High (0.5% annual)

Gnosis Safe	High	High	Medium	High	Medium (gas fees)
Bitcoin P2SH	High	High	Low	Low	Low
Hardware Multi-Sig	High	Very High	Low	Low	Medium (devices)

Each existing solution forces users to compromise on critical dimensions. Centralized solutions sacrifice decentralization and user sovereignty. Blockchain-native solutions present usability challenges and may have limited functionality. This landscape reveals a clear need for multi-signature wallet implementations that maintain decentralization while improving usability and offering flexible, programmable governance mechanisms.

1.3 Why Blockchain is Suitable for Multi-Signature Wallets

Blockchain technology provides unique properties that make it not merely suitable but optimal for implementing secure, transparent, and flexible multi-signature wallet systems.

1.3.1 Trustlessness and Elimination of Intermediaries

Traditional multi-party fund management requires trusted intermediaries—banks, escrow services, or custody providers—who charge fees, introduce counterparty risk, and require users to trust their security practices and operational integrity. Blockchain-based smart contracts eliminate these intermediaries entirely [11].

In a blockchain multi-signature wallet, the rules governing fund access and transaction approval are encoded in transparent, immutable smart contract code. This code executes deterministically across the blockchain network without possibility of intervention, censorship, or manipulation [12]. For example, in a 3-of-5 multi-signature wallet smart contract, it is cryptographically impossible to transfer funds without three valid signatures from authorized addresses, regardless of external pressure, legal orders, or institutional policies.

This trustlessness extends beyond basic transaction execution to all aspects of wallet governance. Owner additions, threshold modifications, and administrative changes are all governed by the same transparent, immutable rules encoded in the smart contract.

1.3.2 Transparency and Complete Auditability

Blockchain provides unprecedented transparency for financial operations. Every transaction, state change, and administrative modification is permanently recorded on the public blockchain with precise timestamps and cryptographic proof of authorization.

Public verifiability means anyone can inspect: the complete smart contract code defining wallet rules, the current wallet configuration including all authorized owners and signature thresholds, the complete transaction history with exact timestamps, and which specific addresses signed which transactions. This creates a permanent, tamper-proof audit trail that cannot be altered or disputed.

For organizations, this transparency provides critical benefits: regulatory compliance through complete transaction records, dispute resolution with cryptographic evidence of authorization, accountability where transaction approval cannot be denied, and trust verification where stakeholders can independently confirm that governance rules are properly followed.

This contrasts sharply with traditional banking systems where account records are private, controlled by the financial institution, and may require legal intervention to access or verify.

1.3.3 Programmable Security Policies and Governance

Smart contracts enable arbitrarily complex security and governance logic that would be prohibitively expensive or impossible to implement in traditional financial systems [13].

Flexible signature thresholds can be implemented as simple M-of-N requirements (e.g., 2-of-3, 3-of-5), weighted signatures where different signers have different voting power, or dynamic thresholds based on transaction characteristics (e.g., small transfers require 2-of-5 signatures while large transfers require 4-of-5).

Time-based controls provide additional security layers: time-locks can require waiting periods for high-value transactions, allowing additional review and potential cancellation; scheduled payments can execute automatically when time conditions are met; and "deadman switch" mechanisms can adjust signature thresholds if no activity occurs for a specified period, facilitating inheritance scenarios.

Spending policies can enforce daily, weekly, or monthly spending limits, set per-transaction maximums, maintain whitelists of approved recipient addresses, or implement hierarchical approval workflows based on transaction amount or type.

Example of complex programmable logic:


```
IF transaction_amount < $10,000  
    THEN require 2-of-5 signatures  
ELSE IF transaction_amount < $100,000  
    THEN require 3-of-5 signatures + 24-hour time-lock  
ELSE  
    THEN require 4-of-5 signatures + 48-hour time-lock + CEO  
signature mandatory
```

Implementing such logic in traditional banking would require extensive custom development, ongoing maintenance costs, and coordination with financial institutions. On blockchain, this becomes simple smart contract code that executes automatically and immutably.

1.3.4 Censorship Resistance and Permissionless Access

Blockchain multi-signature wallets cannot be frozen, seized, or censored by any centralized authority. No government, institution, or individual can prevent authorized signers from accessing funds or executing transactions that meet the defined criteria.

Geographic freedom means users can access and manage funds from anywhere globally, without restrictions based on citizenship, residency, banking relationships, or political climate. Operations proceed 24/7/365 without business hours or holiday interruptions.

Resistance to coercion is built into the multi-signature model: even if some signers are compromised, coerced, or unavailable, the system continues functioning as long as the signature threshold can be met by honest, available signers. No single point of pressure can compromise the entire system [14].

This contrasts with traditional finance where bank accounts can be frozen by government order, international transfers can be blocked or delayed, and access can be denied based on regulatory or policy decisions.

1.3.5 Cryptographic Security and Immutability

Blockchain provides mathematical security guarantees through cryptographic foundations. **Immutable transaction history** ensures that once confirmed on the blockchain, transactions cannot be altered, reversed (except through new transactions), deleted, or hidden. All operations are cryptographically signed using elliptic curve cryptography (ECDSA for Ethereum), providing unforgeable digital signatures that prove authorization without revealing private keys.

Smart contract immutability means that once deployed, contract code cannot be altered unless explicit upgrade mechanisms are built in. This provides certainty that the rules governing fund access remain constant and cannot be changed unilaterally by any party.

1.3.6 Cost Efficiency

Despite transaction fees (gas costs on Ethereum), blockchain multi-signature solutions can be significantly more cost-effective than traditional alternatives.

No ongoing custody fees: Unlike custodial services charging 0.5-1% annually, blockchain wallets incur only one-time deployment costs and per-transaction gas fees with no minimum balances or monthly maintenance charges.

Example cost comparison for managing \$1 million over one year:

- Coinbase Custody: ~\$10,000 annual fee (1%)
- BitGo: ~\$5,000 annual fee (0.5%)
- Gnosis Safe: ~\$500 deployment + \$50-100 per transaction

For organizations making infrequent transactions, blockchain solutions offer substantial savings while maintaining superior security and transparency.

1.3.7 Composability with Decentralized Finance (DeFi)

Blockchain smart contracts seamlessly integrate with the broader DeFi ecosystem [15]. Multi-signature wallets can directly interact with decentralized exchanges, lending protocols, yield farming platforms, and governance systems without requiring intermediary approval or coordination.

This **composability** enables organizations to manage funds collectively while participating in DeFi opportunities: lending assets to earn yield, providing liquidity to decentralized exchanges, staking tokens in proof-of-stake networks, and voting in DAO governance with collective decision-making.

Traditional finance typically does not allow joint accounts to directly participate in investment products, or requires separate authorization processes for each activity.

1.3.8 Alignment with Blockchain Philosophy

Multi-signature wallets embody core blockchain principles: **decentralization** through distributed control among multiple parties rather than concentration in single entities; **trustlessness** with security derived from cryptography and mathematics rather than institutional trust; **transparency** through publicly verifiable rules and transactions; **self-sovereignty** where users maintain direct asset control without intermediaries;

and **permissionless** operation where anyone can create and use multi-signature wallets without requiring institutional approval.

1.4 Scope and Objectives

This project aims to design, implement, and evaluate a fully functional multi-signature wallet system on the Ethereum blockchain, specifically targeting the security and governance limitations of single-signature wallets identified in the preceding sections.

At the smart contract level, three modular Solidity contracts will be developed and deployed on the Ethereum test network. The core `MultiSigWallet.sol` contract will manage the M-of-N threshold logic and maintain transaction state, `OwnerManager.sol` will handle dynamic governance operations such as adding and removing authorized signers, and `TransactionManager.sol` will govern the complete transaction lifecycle from proposal submission to final execution. All contracts will incorporate OpenZeppelin's `ReentrancyGuard` and access control libraries, and will implement nonce-based replay protection compliant with EIP-155.

At the off-chain application level, a Python backend built on `Web3.py` will handle contract interaction, event listening, and cryptographic signature management. To further reduce on-chain gas costs, the system will explore off-chain signature aggregation using EIP-712 typed structured data, allowing multiple signatures to be collected off-chain and submitted in a single batched transaction.

At the user interface level, a Streamlit dashboard will provide wallet owners and co-signers with an accessible interface to propose transactions, review and approve pending requests, monitor real-time wallet activity, and manage ownership configurations — all without requiring direct low-level blockchain interaction.

The evaluation phase will validate the system across three dimensions: functional correctness through automated unit and integration tests targeting over 90% code coverage on a local Hardhat network, security verification through Slither and Mythril static analysis tools supplemented by a manual audit of the attack vectors defined in Section 2.3, and gas efficiency benchmarking comparing the proposed implementation against Gnosis Safe as the current industry standard. The ultimate objective is to deliver a working prototype that demonstrates decentralized, transparent, and programmable multi-party asset management without reliance on any centralized intermediary.

The properties outlined above — trustlessness, transparency, programmable governance, and censorship resistance — collectively establish blockchain as not merely a viable but an optimal substrate for multi-signature wallet systems. However, translating these theoretical advantages into a concrete, secure, and deployable implementation requires a precise formulation of the problem space. The following section formalizes the stakeholder model of the proposed multi-signature wallet,

defines the operational workflow spanning from wallet creation through transaction execution, and identifies the specific attack vectors — including signature replay, reentrancy, and quorum manipulation — that the smart contract architecture must actively defend against.

2. PROBLEM FORMULATION

Details that need to be taken into consideration when defining a multi-signature problem are: The technical architecture of the wallet system, stakeholder roles, operational workflow, and critical security threats faced by smart contract-based systems [17].

2.1 Stakeholders and Roles

System security and governance rely on interaction between stakeholders with different levels of authority. All roles have permissions that cover certain functions on the smart contract. The management of these roles is not static, it is designed to be as dynamic as the use of the wallet.

The process of assigning and changing roles follows a strict governance protocol where any change to the mapping of owners requires a process that passes the current threshold check. This process, which ensures the decentralization of administrative updates, can prevent a single compromised administrative key from unilaterally allowing malicious addresses. Where,

Wallet Owners/Creators: The most authorized stakeholders who set up the wallet. They determine the initial parameters (M-of-N threshold, initial list of signatories).

Co-Signers: Authorized addresses that provide the digital signatures required for transfers to occur. They are part of the set "N" in the M-of-N structure [17].

Transaction Proposers: Parties who offer to spend wallet balance or trigger a smart contract. Usually one of the owners assumes this role, but authority to bid on specific non-system addresses can also be defined.

Beneficiaries: These are the destination addresses to which funds are transferred when the transaction is confirmed. These can be personal wallets or exchanges.

External Contracts: DeFi protocols or other smart contracts. The multi-signature wallet can interact with these external systems as an "administrator" or "investor".

Thus, the system may implement a "weight-based" permission hierarchy in which signatures from certain high-level stakeholders are counted as more weighted, but the default implementation treats all signatures as equal peers to maintain simplicity and minimize attack surfaces.

Permission Hierarchy and Interactions:

As illustrated in Figure 1, the strict hierarchy in the system can be described in terms of certain functions. Proposer sends the transaction to the pool, but this transaction is in pending status. Co-Signers review and approve transactions. When this number of

confirmations reaches the specified threshold (M), any stakeholder can process the transaction into the blockchain with the execute command. Role changes (adding/removing new owners) can only be made with the approval of M signatures. In this way, a single person is prevented from seizing the wallet.

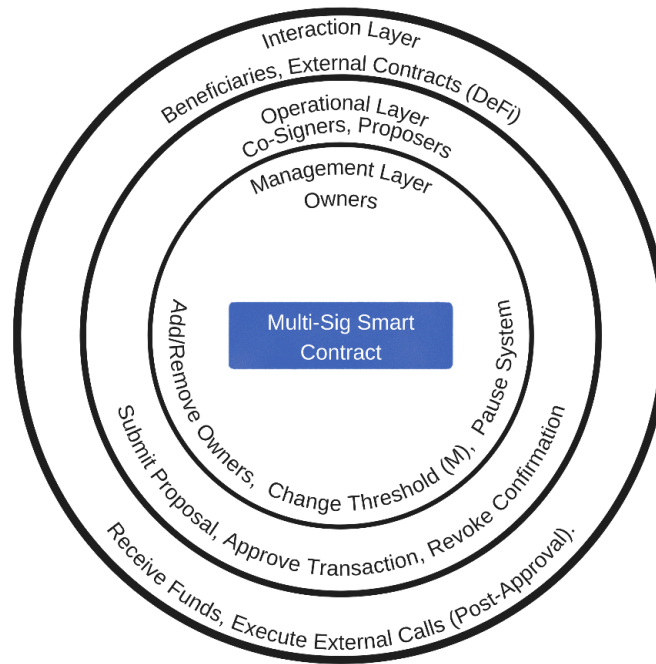


Figure 1: Stakeholder Permission Hierarchy. The diagram illustrates a tiered access model where critical administrative functions (Inner Circle) require the highest consensus, while operational tasks and external interactions occupy the outer layers.

2.2 System Interactions and Workflow

System operation consists of three main stages: installation, transaction cycle and administrative operations. Beyond basic execution, this system relies on event-driven architecture to provide off-chain transparency. All phases of the interaction, from the first submission to each confirmation, emit a specific Ethereum event. These events are used as a decentralized audit log that front-end interfaces and indexing services, such as The Graph, can monitor in real time. This allows beneficiaries and stakeholders to track the progress of a transaction without manually querying the contract status, bridging the gap between low-level blockchain data and user-level operational awareness.

2.2.1 Wallet Creation Flow

The wallet lifecycle begins with the distribution of the contract to the network. The founding partners enter parameters such as the list of validating addresses, the

minimum number of signatures required for transaction approval, and the loading of the first balance of the contract into the system through the function called constructor.

2.2.2 Transaction Flow

This process has three steps: submission, review - approval and execution. In the first step, an official creates the offer by specifying the target address, amount and data (function call) content, if any. Each quote is assigned a unique transactionId. In the review and approval step, signatories review the transaction through the wallet interface or directly through the contract. If transaction verification is deemed appropriate, the confirmTransaction function is called. Finally, in the execution step, the executeTransaction function is triggered when the number of confirmations is equal to or greater than M. The contract sends the funds to the target address and provides transparency by broadcasting the Execution event.

2.2.3 Administrative Operations

Administrative functions are used to maintain the dynamic structure of the wallet. In case the owners lose their keys or leave the team, the addOwner, removeOwner or replaceOwner functions are used. Additionally, the emergencyPause mechanism can be activated to stop operations if a vulnerability is detected in the system. All operations are subject to multiple approval mechanisms, such as a standard funds transfer.

2.3 Security Concerns and Attack Vectors

Since smart contracts are public and irreversible, proactive measures are taken against attack vectors.

Attack Vector 1 - Signature Replay Attacks: An attacker can capture the signature of a valid transaction made in the past and send it back to the network to make repeated fund withdrawals. The precaution to be taken as a result of this attack is to use a unique counter (nonce) for each transaction and include in the hash which contract address and chain ID the transaction is valid for. This measure can be included within the scope of the EIP-155 standard [16].

Attack Vector 2 - Reentrancy Attacks: When an external contract receives funds from the wallet, it can empty the wallet by calling the withdraw function again without giving the wallet the opportunity to update its balance. In response, the Check-Effects-Interactions pattern is applied. First the status is updated, then the external call is made. Additionally, locking mechanisms (mutex) such as ReentrancyGuard are used [16].

Attack Vector 3 - Front-Running & MEV Attacks: In a public mempool environment, malicious actors or MEV (Maximal Extractable Value) bots can observe a pending multi-sig transaction and attempt to execute their own transactions with higher gas fees to profit from the price impact or state change. While the multi-signature requirement mitigates many of these risks by ensuring only authorized data is executed, administrative changes remain vulnerable to front-running. To counter this, the system can utilize a "Commit-Reveal" scheme for critical parameters or implement a mandatory Time lock mechanism. A time lock ensures that once an admin change is approved, it must wait for a predefined period (e.g., 24 hours) before it can be finalized, giving honest stakeholders enough time to react if the proposed change was made under duress or by a compromised quorum.

Attack Vector 4 - Social Engineering: It includes attacks such as infiltrating signatories' computers or confirming a false transaction through a fake interface. On the other hand, Hardware Wallet integration and simulation tools that clearly show transaction details are used as precautions.

Attack Vector 5 - Quorum Manipulation: Some of the wallet owners cooperate and lower the threshold value (M) and gain control. The precaution against this attack is to limit the minimum threshold value by hard-coded at the contract level (for example, M can never be less than $N/2$) and to seek the approval of all owners in threshold changes [16].

The technical mitigations described above are only effective when paired with strict operational security (OpSec). As the security of the multi-signature wallet is ensured, the physical and digital security of each signer becomes a new perimeter. If M number of signed protocols all use insecure wallets on the same platform, the decentralized structure of the contract becomes ineffective. Therefore, the system design aims to avoid problems caused by single software vulnerabilities by encouraging a diverse processor strategy.

Table 2: Summary of Security Threats and Proposed Technical Mitigations

Attack Vector	Nature of Threat	Mitigation Strategy
Signature Replay	Reusing a valid signature on a different transaction or network.	Nonce-based tracking and ChainID (EIP-155) binding.

Reentrancy	Malicious contracts calling back into the wallet before state updates.	"Checks-Effects-Interactions" pattern and Mutex locks.
Front-Running	MEV bots executing transactions ahead of authorized signers.	Implementation of Time-locks for sensitive admin changes.
Quorum Manipulation	Collusion to lower the M-threshold for unauthorized access.	Hard-coded minimum thresholds and multi-sig required for logic updates.

2.4 Scalability and Gas Optimization:

A significant challenge in multi-signature design is the linear increase in gas costs as the number of signers (N) and the threshold (M) grow. Each confirmTransaction call requires a separate blockchain interaction, which can become prohibitively expensive during periods of high network congestion. To optimize this, the proposed architecture explores the use of "Off-chain Signature Aggregation". By collecting EIP-712 typed signatures off-chain and submitting them in a single batch transaction, the wallet can significantly reduce the cumulative gas fees, making the governance process more sustainable for large-scale Decentralized Autonomous Organizations (DAOs) [17].

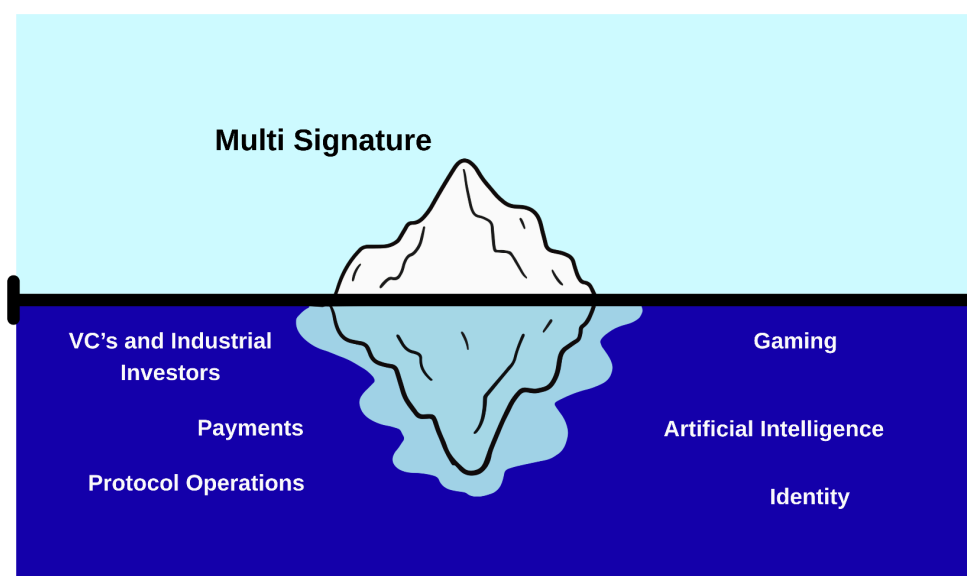


Figure 2: The Iceberg Model of Multi-Signature Utility. While the visible part represents the basic multisig functionality, the underlying structure supports complex interactions including Protocol Operations, Identity management, and automated payments

As depicted in the "Iceberg Model" in Figure 2, the multi-signature implementation serves as a foundational security layer for diverse decentralized applications. For instance, in Protocol Operations, the wallet doesn't just hold funds; it manages the administrative keys of other smart contracts, ensuring that protocol upgrades require a decentralized consensus. Similarly, in the realm of **Identity and Gaming**, a multi-sig structure can represent a shared collective identity or a guild treasury, providing a robust framework that extends far beyond simple peer-to-peer transfers.

Having formally characterized the stakeholder roles, transaction lifecycle, security threats, and scalability constraints, it is now possible to define a concrete technical response to each identified challenge. The proposed architecture in the following section directly maps to these requirements: the modular smart contract structure addresses the governance and attack surface concerns of Section 2.3, while the off-chain signature aggregation strategy directly responds to the gas optimization problem identified in Section 2.4.

3. PROPOSED METHOD / APPROACH

To address the vulnerabilities of single-signature wallets and the centralization risks inherent in custodial services, we propose a modular, smart contract-based multi-signature wallet architecture. This approach emphasizes decentralization, operational transparency, and robust security through a clearly defined on-chain and off-chain ecosystem.

3.1 Smart Contract Architecture

To address the architectural needs of the multi-signature wallet, we propose a modular smart contract design rather than a monolithic structure, ensuring long-term upgradeability, reduced code complexity, and a minimized attack surface. The system's logic is distributed across three primary, interconnected components based on their distinct responsibilities. The core component functions as the primary state repository and entry point, securely maintaining essential data such as the roster of authorized owners, the required signature threshold, and the queue of pending transactions. A secondary administrative component is exclusively dedicated to the wallet's dynamic governance, securely facilitating modifications to the owner list and enforcing strict validation rules to ensure the signature threshold remains within safe operational limits. Finally, a transactional component oversees the complete lifecycle of fund transfers and external interactions, meticulously tracking cryptographic approvals and guaranteeing that transactions are only executed once the predefined consensus is achieved.

To reinforce the security of these relationships, the contracts will utilize inheritance structures leveraging battle-tested libraries, specifically incorporating OpenZeppelin's implementation of access controls and security guards [18].

3.2 Platform Components

The system is divided into two distinct layers to balance operational efficiency with blockchain immutability:

- **On-Chain Components:** All critical logic, state storage, and transaction execution occur on the Ethereum blockchain. The Solidity smart contracts enforce the M-of-N threshold and emit standardized events (e.g., SubmitTransaction, ConfirmTransaction, ExecuteTransaction, OwnerAddition) for every state change. This event-driven architecture is crucial for maintaining the public auditability discussed in the problem formulation.
- **Off-Chain Components:** To bridge the gap between low-level blockchain infrastructure and end-user experience, an off-chain application will be developed using Python. Utilizing the **Web3.py** library, the backend will

interact with the blockchain to construct transaction proposals, fetch pending requests, and manage cryptographic signatures. For the user interface, a Python-based framework such as **Streamlit** (or **Flask**) will be employed to provide stakeholders with a clean, interactive dashboard to review and sign transactions seamlessly [19].

3.3 Tools and Frameworks

The development, testing, and deployment pipeline relies on industry-standard frameworks tailored for a Python-centric workflow:

- **Solidity (0.8.x):** The primary programming language for the smart contracts, chosen for its built-in overflow and underflow protection [20].
- **Web3.py:** A comprehensive Python library for interacting with Ethereum. It will be the core tool for the off-chain application to read contract states, listen to events, and send signed transactions to the network [19].
- **Streamlit / Flask:** Python-based frameworks used to rapidly develop the interactive frontend dashboard, allowing stakeholders to interact with the smart contract without needing deep technical knowledge [21].
- **Hardhat (or Brownie):** While the application logic is Python-based, Hardhat (or the Python-equivalent Brownie) will be utilized for the local blockchain simulation, smart contract compilation, and initial deployment scripting [22].
- **OpenZeppelin Contracts:** We integrate OpenZeppelin's secure libraries, specifically utilizing ReentrancyGuard to mitigate the callback attacks identified in Section 2.3, ensuring the "Checks-Effects-Interactions" pattern is strictly enforced [18].

3.4 Key Features

The proposed solution directly addresses the limitations of current systems through several key features:

1. **Configurable M-of-N Threshold:** Dynamic governance allowing authorized updates to the signature threshold as organizational needs scale.
2. **Transparent Proposal Workflow:** A verifiable, multi-step transaction approval process with real-time on-chain status tracking.
3. **Mandatory Time-Locks:** Integration of time-delays for high-value transactions and critical administrative changes, mitigating front-running and coercion risks.
4. **Gas Optimization:** Utilization of off-chain signature aggregation to minimize the linear gas cost scaling traditionally associated with multi-signature wallets.

The architectural decisions and security mechanisms described above are only as strong as the verification methodology used to validate them. Given that smart contract vulnerabilities are irreversible by nature, a rigorous, multi-layered evaluation strategy is not optional but essential. The following section defines the specific criteria and methodologies by which each component of the proposed system will be tested, audited, and benchmarked against existing solutions.

4. EVALUATION PLAN

Ensuring the reliability of financial infrastructure requires an evaluation strategy that leaves no room for functional or security flaws. The evaluation plan is divided into three rigorous domains: functional correctness, security verification, and gas efficiency analysis.

4.1 Functional Correctness Testing

Drawing upon rigorous software testing methodologies, the system's functional correctness will be validated through an extensive suite of automated tests. The absolute success criterion for this phase is achieving greater than 90% code coverage.

- **Unit Tests:** Every individual function within the smart contract modules will be isolated and tested. This includes verifying wallet creation with diverse M-of-N configurations, ensuring `submitTransaction` correctly logs state, and validating that the `OwnerManager` strictly rejects invalid threshold updates (e.g., attempting to set a 4-of-3 threshold).
- **Integration Tests:** We will simulate the complete lifecycle of complex transactions on a local Hardhat network. This includes evaluating multi-owner scenarios (e.g., 2-of-3, 3-of-5 configurations) from initial proposal to final execution. Critical edge cases will be systematically tested, such as handling duplicate confirmations from a single address, rejecting invalid signatures, and ensuring that revoked confirmations accurately update the transaction state.

4.2 Security Evaluation

Because smart contracts handle financial assets immutably, ensuring the computational security of the cryptographic primitives and the logical security of the code is paramount.

- **Static Analysis:** We will employ automated vulnerability detection tools to scan the abstract syntax tree of our Solidity code. **Slither** will be used to detect common logic flaws, variable shadowing, and unprotected functions [22]. **Mythril** will be utilized for symbolic execution to uncover deeper logical bugs within the EVM bytecode [23].
- **Manual Security Review:** Automated tools will be supplemented with a strict manual audit focusing on the attack vectors outlined in Section 2.3. The review will systematically verify:
 - *Reentrancy:* Confirming `ReentrancyGuard` and state-update ordering.

- *Access Control*: Ensuring only the wallet itself (via multi-sig consensus) can call administrative functions like addOwner.
- *Signature Protection*: Validating the computational security of the ECDSA signature verification against replay attacks by checking nonce increments and EIP-155 Chain ID bindings.
- **Security Test Cases**: We will author adversarial test scripts attempting to execute unauthorized transactions, bypass the M-of-N threshold, and execute front-running attacks against administrative changes to prove the time-lock mitigations are effective.

4.3 Gas Efficiency Analysis

To ensure the solution is practically viable and competitive with existing options like Gnosis Safe, we will conduct a detailed gas efficiency analysis.

- **Metrics Tracked**: We will measure and document the exact gas consumption for: (1) Initial one-time contract deployment, (2) Submitting a transaction proposal, (3) Confirming a transaction, and (4) Executing the final transaction.
- **Optimization Strategies**: We will evaluate the success of our gas optimization techniques, such as batching operations, minimizing expensive storage updates (using calldata over memory), and utilizing event emissions in place of on-chain storage for non-critical historical data.
- **Success Criteria**: The system will be considered successful if 100% of functional tests pass, zero critical or high-severity vulnerabilities are found in the final audit, and the cumulative gas cost for a standard 2-of-3 transaction lifecycle is proven to be competitive with current industry baselines.

5. CONCLUSION

This project proposal covers the development and evaluation of a modular smart contract-based multi-signature wallet system on the Ethereum blockchain. Given the security vulnerabilities of single-signature wallets and the centralization risks of existing solutions, the proposed system is expected to offer a meaningful alternative for multi-party asset management.

The expected deliverables at the end of the project can be summarized as follows: a working smart contract prototype deployed on the Ethereum test network that fully enforces M-of-N threshold logic and passes security testing; a functional Streamlit dashboard through which wallet owners and co-signers can manage transactions without requiring technical blockchain expertise; and a comprehensive testing and security audit report including a comparative gas efficiency analysis against Gnosis Safe.

From a security standpoint, the mitigations implemented against known attack vectors such as signature replay and reentrancy are expected to be verified through Slither and Mythril analysis, with the target of zero critical vulnerabilities achieved. From a gas optimization standpoint, the EIP-712 based off-chain signature aggregation approach is expected to yield a measurable cost reduction compared to the standard on-chain confirmation process.

From a broader perspective, this work is expected to serve as a practical reference point for multi-signature wallet usage in areas such as DAO governance, institutional cryptocurrency management, and digital inheritance planning. Successful completion of the project will concretely demonstrate that secure and accessible multi-party asset management is achievable without reliance on any centralized intermediary.

REFERENCES

- [1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.
- [2] A. Narayanan, J. Bonneau, E. Felten, A. Miller, and S. Goldfeder, *Bitcoin and Cryptocurrency Technologies*. Princeton University Press, 2016.
- [3] MicroStrategy, "Bitcoin holdings update," *Investor Relations*, 2023.
- [4] K. Werbach, "Trust, but verify: Why the blockchain needs the law," *Berkeley Technology Law Journal*, vol. 33, no. 2, pp. 487-550, 2018.
- [5] Coinbase, "Coinbase Custody: Institutional-grade cryptocurrency custody," 2023.
- [6] V. Buterin, "The meaning of decentralization," *Medium*, Feb. 2017.
- [7] A. M. Antonopoulos, *Mastering Bitcoin: Programming the Open Blockchain*, 2nd ed. O'Reilly Media, 2017.
- [8] Gnosis, "Gnosis Safe: The most trusted platform to manage digital assets," 2023.
- [9] BIP 16, "Pay to Script Hash," Bitcoin Improvement Proposal, Jan. 2012.
- [10] J. Bonneau, A. Miller, J. Clark, A. Narayanan, J. A. Kroll, and E. W. Felten, "SoK: Research perspectives and challenges for Bitcoin and cryptocurrencies," *IEEE Symposium on Security and Privacy*, pp. 104-121, 2015.
- [11] N. Szabo, "Smart contracts: Building blocks for digital markets," *EXTROPY: The Journal of Transhumanist Thought*, vol. 16, 1996.
- [12] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Yellow Paper*, 2014.
- [13] M. Crosby, P. Pattanayak, S. Verma, and V. Kalyanaraman, "Blockchain technology: Beyond Bitcoin," *Applied Innovation Review*, vol. 2, pp. 6-19, 2016.
- [14] J. Abadi and M. Brunnermeier, "Blockchain economics," *NBER Working Paper* No. 25407, 2018.
- [15] F. Schär, "Decentralized finance: On blockchain- and smart contract-based financial markets," *Federal Reserve Bank of St. Louis Review*, vol. 103, no. 2, pp. 153-174, 2021.
- [16] Antonopoulos, A. M., & Wood, G. (2018). *Mastering Ethereum: Building Smart Contracts and DApps*. O'Reilly Media.
- [17] Buterin, V. (2014). *A Next-Generation Smart Contract and Decentralized Application Platform*. Ethereum Whitepaper

- [18] OpenZeppelin, "OpenZeppelin Contracts: Secure Smart Contract Development," OpenZeppelin Documentation, 2023. [Online]. Available: <https://docs.openzeppelin.com/contracts>.
- [19] Ethereum Foundation, "Web3.py: A Python library for interacting with Ethereum," Web3.py Documentation, 2023. [Online]. Available: <https://web3py.readthedocs.io/>.
- [20] Solidity Team, "Solidity ^0.8.0 Documentation: Checked or Unchecked Arithmetic," Ethereum Foundation, 2021.
- [21] Streamlit Inc., "Streamlit: The fastest way to build and share data apps," 2023. [Online]. Available: <https://streamlit.io/>.
- [22] Nomic Foundation, "Hardhat: Ethereum development environment for professionals," 2023. [Online]. Available: <https://hardhat.org>.
- [23] J. Feist, G. Grieco, and A. Groce, "Slither: a static analysis framework for smart contracts," in *Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain*, 2019, pp. 8-15.
- [24] ConsenSys, "Mythril: Security analysis tool for EVM bytecode," ConsenSys Security, 2023.